Bitácora del Proyecto

App de comparación de precios — Falabella, Ripley, Paris y Ahumada

---

Este documento registra, sesión por sesión, los pasos efectivamente realizados durante el desarrollo de la aplicación móvil: una app que permite sacar una foto de un producto, identificarlo automáticamente y comparar su precio entre cuatro tiendas chilenas. Cada sesión resume lo que se hizo, los problemas encontrados y cómo se resolvieron.

Sesión 1 — Preparación de cuentas y estructura inicial del proyecto

16 de septiembre de 2026

Se revisó el plan de preparación recomendado antes de comenzar a programar. Se confirmó que ya estaban creadas las cuentas de Gmail del negocio, Anthropic Console, Railway y Apify, quedando pendiente únicamente la cuenta de GitHub.

Al intentar crear la cuenta de GitHub, el navegador del usuario mostró un bloqueo de seguridad por "actividad inusual". Se verificó, probando la misma página con otro navegador, que el problema era del navegador o la red del usuario y no de GitHub; la cuenta se creó exitosamente desde ahí.

Con las cinco cuentas listas, se armó la estructura inicial del proyecto (carpeta "PDN project"), incluyendo un backend en Python con FastAPI con dos funciones principales — identificar un producto a partir de una foto usando Claude, y buscar su precio en las cuatro tiendas usando Apify —, además de un archivo de configuración de ejemplo para las claves (.env.example), reglas de exclusión para Git (.gitignore) y un README con instrucciones paso a paso para continuar.

Sesión 2 — Backend funcionando localmente

17 de septiembre de 2026

Se instaló Git (no estaba presente en el equipo) y se configuró la identidad de los commits. Se inicializó el repositorio local, se conectó al repositorio vacío creado en GitHub y se subió la estructura inicial del proyecto.

Se creó el entorno virtual de Python para el backend. Surgió un problema de permisos de ejecución de scripts en PowerShell (política de ejecución de Windows), que se resolvió habilitando la política adecuada para el usuario. La instalación de dependencias falló inicialmente porque las versiones fijadas en requirements.txt no tenían instaladores compatibles con la versión de Python instalada (3.14) e intentaban compilarse desde código fuente, lo que fallaba por falta de herramientas de compilación de Visual Studio; se resolvió flexibilizando las versiones del archivo de dependencias.

Se explicó cómo obtener el token de API de Apify y los identificadores de los actores de scraping de Falabella, Ripley, Paris y Ahumada, y se cargaron junto con la clave de Anthropic en el archivo de configuración (.env) del backend.

Con las claves configuradas, se levantó el servidor local del backend (uvicorn) y se confirmó que la documentación interactiva de la API (/docs) respondía correctamente, cerrando la sesión con el backend funcionando de punta a punta en el computador.

Sesión 3 — Pantallas reales de la app y depuración de punta a punta

18 de septiembre de 2026

Se instaló Flutter y el Android SDK a través de Android Studio (incluyendo las "Command-line Tools", que no vienen instaladas por defecto), y se generó el proyecto Flutter dentro de la carpeta "mobile", confirmando que corría correctamente en Chrome.

Se reemplazó la pantalla de ejemplo de Flutter por las pantallas reales de la app: una pantalla de inicio para tomar o elegir una foto, y una pantalla de resultados que muestra la comparación de precios entre las cuatro tiendas, destacando la más barata. Se agregaron los permisos de cámara e internet necesarios en Android, y se habilitó CORS en el backend para permitir que la app le hiciera peticiones.

Durante las pruebas surgió un problema recurrente de sincronización de OneDrive que interfería con la compilación de Flutter (bloqueos de archivos y de permisos, e incluso una reversión silenciosa de un archivo ya editado). Se decidió mover la carpeta completa del proyecto fuera de OneDrive, lo que resolvió los problemas de compilación.

Ya con la app corriendo, se depuró el flujo completo de extremo a extremo, encontrando y resolviendo en orden: un error de "tipo de imagen inválido" porque la app no indicaba correctamente el tipo de archivo subido; un error de "no se pudo procesar la imagen" que resultó ser porque las fotos de iPhone vienen en formato HEIC (no soportado por Claude), solucionado agregando conversión automática a JPEG en el backend; un error en la búsqueda de precios porque los actores de Apify esperaban el campo "term" en vez de "search"; y finalmente un error porque la librería de Apify, en su versión instalada, devuelve un objeto en vez de un diccionario.

En el camino se presentó también una dificultad repetida en la que los cambios guardados en el backend no llegaban a reflejarse de forma confiable en el archivo del usuario. Se resolvió verificando el contenido de forma independiente directamente en PowerShell y, en los casos más persistentes, reemplazando el archivo completo a mano en el editor.

La sesión cerró con una prueba exitosa de principio a fin: se sacó una foto de un tensiómetro digital, la app lo identificó correctamente ("Tensiómetro digital de brazo Beurer BM 27") y mostró la comparación real de precios: Paris a $6.990 (la más barata, destacada), Ahumada a $23.990 y Ripley a $99.990, con Falabella indicando que no encontró el producto en su catálogo.

Pendiente para próximas sesiones:
Revisar por qué Falabella no devuelve resultados para ciertos productos, y desplegar el backend en Railway para que la app funcione sin depender de que el computador del usuario esté encendido.

Sesión 4 — Falabella funcionando, fotos de producto, links de compra y mejor coincidencia de resultados

25 de septiembre de 2026

Se retomó el problema pendiente de Falabella sin resultados. Revisando el error real entregado por el actor de Apify, se identificó un bug propio del actor: al traer varias páginas de resultados, Falabella a veces cambia el conteo de paginación entre una página y otra, y el actor aborta toda la corrida con una excepción. Se resolvió limitando el actor a una sola página de resultados, con lo que Falabella pasó de fallar siempre a devolver resultados de forma consistente (48 productos en la primera prueba).

Con Falabella ya devolviendo resultados, se notó que no mostraba precio. Se detectó que Falabella no entrega un único precio "correcto": según el producto, el precio real puede estar en distintos campos, y varios de ellos corresponden al precio "antes de descuento" tachado en la página, no al precio final. La primera corrección tomaba el campo equivocado en algunos casos (por ejemplo, mostraba $27.990 en vez de los $17.990 reales de una promoción vigente). Se corrigió juntando todos los precios que trae cada producto y quedándose siempre con el más bajo, que en la práctica es el precio que efectivamente se cobra.

Se hizo una primera prueba de extremo a extremo directamente en Chrome de escritorio (en vez del celular), subiendo una foto guardada del producto en lugar de sacarla en vivo, ya que en un navegador de escritorio no hay forma de invocar la cámara del equipo de la misma manera que en un celular.

Se agregaron miniaturas del producto encontrado en cada tienda dentro de la pantalla de resultados. En la primera versión solo se veía la imagen de Falabella: se descubrió que Paris, Ripley y Ahumada bloquean que su imagen se cargue directamente desde fuera de su propio sitio web (una protección habitual contra el uso no autorizado de sus imágenes). Se resolvió agregando un endpoint en el backend que descarga la imagen por su cuenta y se la entrega a la app, evitando así esa restricción. De paso, se agregó la opción de tocar la miniatura para verla ampliada con zoom.

Se detectó que, para ciertas búsquedas, el primer resultado de una tienda no tenía relación con el producto buscado (por ejemplo, una bicicleta infantil en vez de una cámara digital infantil, porque ambos productos comparten palabras como "infantil" o "niño"). Se agregó una lógica de relevancia que compara las palabras de la búsqueda contra el nombre de cada producto devuelto por la tienda y se queda con el más parecido, en vez de tomar ciegamente el primero del listado; si ningún resultado comparte ninguna palabra con la búsqueda, la tienda queda marcada directamente como "no se encontró un producto relevante".

Se agregó un botón "Comprar" en cada tienda con resultado, que abre el link directo del producto en su página para poder comprarlo ahí mismo.

Al probar el botón "Tomar foto" en Chrome de escritorio, se notó que en vez de abrir una cámara abre el explorador de archivos. Se confirmó que es una limitación de cómo funciona ese mecanismo en navegadores de escritorio (en un celular real sí abre la cámara nativa directamente, como ya había ocurrido en la Sesión 3). Se decidió seguir probando con imágenes guardadas por ahora y dejar la prueba con cámara real para una próxima sesión.

Durante la sesión volvió a repetirse, varias veces, la dificultad ya conocida de sincronización de archivos hacia el computador del usuario (cambios que no llegaban a guardarse, o llegaban incompletos o desactualizados). Se siguió resolviendo verificando cada cambio de forma independiente antes de continuar, y reintentando cuando la verificación mostraba que no había llegado bien.

Pendiente para próximas sesiones:
Probar el flujo completo con la cámara real en un celular (o evaluar si vale la pena agregar captura de cámara web para poder seguir probando cómodamente desde el computador), y desplegar el backend en Railway para que la app funcione sin depender de que el computador del usuario esté encendido.

Sesión 5 — Despliegue del backend en Railway

7 de octubre de 2026

Se retomó el pendiente de desplegar el backend para que la app deje de depender de que el computador del usuario esté encendido. Se verificó que el proyecto ya tenía listo el archivo Procfile (con el comando de arranque de uvicorn escuchando en el puerto asignado por la plataforma) y que el .gitignore excluía correctamente el archivo .env, de modo que las claves no se subieran a GitHub.

Se subió todo el trabajo acumulado al repositorio de GitHub (git add, commit y push). Al hacer el commit apareció varias veces el mensaje "Deletion of directory ... failed. Should I try again?": era el mantenimiento automático de Git, que no podía borrar una carpeta interna bloqueada por Windows; no afectaba al commit y se resolvió respondiendo "n".

En Railway se creó un proyecto a partir del repositorio, se configuró la carpeta raíz del servicio como "backend" (porque el backend está en una subcarpeta del repo) y se cargaron como variables de entorno la clave de Anthropic, el token de Apify y los identificadores de los cuatro actores de tiendas. Esas claves las ingresó el propio usuario directamente en Railway. Al generar el dominio público, Railway pidió indicar el puerto; se fijó la variable PORT en 8000 y se usó ese mismo valor, para que el puerto de la aplicación y el del dominio coincidieran.

Se confirmó en los registros del servicio que uvicorn arrancó correctamente y que la ruta /health respondía 200 OK en la URL pública. Luego se cambió la dirección del backend en la configuración de la app (config.dart) para que apunte a Railway en lugar de al computador local, y se repitió la prueba de punta a punta desde Chrome con el servidor local apagado: la identificación del producto, la comparación de precios, las miniaturas y los botones "Comprar" funcionaron sin problemas.

En los registros de Railway apareció un único error de imagen: una miniatura de Ripley devolvía 403 porque la URL entregada por el actor venía truncada (terminaba en un punto, sin la extensión del archivo). Se agregó al backend un respaldo: si la URL de la imagen no termina en una extensión de archivo válida, se busca otra imagen completa dentro de los datos crudos del producto (imagen principal, lista de imágenes o variantes) y se usa esa. La lógica se probó con casos de prueba y el archivo se guardó en el computador del usuario; quedó pendiente subirlo a GitHub para que Railway lo despliegue y confirmar el resultado en la app.

Pendiente para próximas sesiones:

Subir a GitHub el respaldo de imagen de Ripley (git add, commit y push) y confirmar en la app que ya se ve la miniatura; probar el flujo completo con la cámara real en un celular (o evaluar agregar captura de cámara web para probar desde el computador); y, antes de publicar la app, restringir el CORS del backend (hoy permite cualquier origen) y el endpoint de imágenes, que por ahora acepta cualquier URL.

Bitácora generada a partir del registro de trabajo con Claude.